

M1 – Technical Understanding & Research Framing: Low Power Scheduling for High-Level Synthesis

Rionela Kovaci
HW/SW Codesign Seminar
Hochschule Hamm-Lippstadt (HSHL)
Hamm, Germany

Abstract—This document is the Milestone 1 (M1) deliverable for the seminar topic *Low Power Scheduling for High-Level Synthesis*. The assigned reference [1] presents a scheduling method called the *Power Scheduler*, which reduces energy consumption during High-Level Synthesis (HLS) by grouping mutually exclusive execution paths into switchable partitions controlled by guard signals. This briefing summarises the technical problem, the main idea, the algorithm steps, key assumptions, and open questions.

I. PROBLEM STATEMENT & MOTIVATION

- Portable and wireless devices such as mobile phones need to run on small batteries for as long as possible, while at the same time offering more features than before.
- Power is lost in two main ways inside a chip:
 - **Dynamic power** – transistors draw current every time they switch state. The formula is: $P_{\text{dyn}} = \frac{1}{2} C_L V_{dd}^2 f_{clk}$
 - **Static power** – a small leakage current always flows through transistors, even when nothing is switching.
- HLS is the design step that translates a behavioural description written in C or VHDL into a hardware netlist at Register-Transfer Level (RTL). If power is not considered here, later fixes are costly and less effective.
- Standard HLS tools only optimise for area A and timing D ; power P is not treated as a design goal.

II. CORE TECHNICAL IDEA

- The proposed tool is called the **Power Scheduler**. It adds power awareness to the scheduling step of HLS.
- The main observation is: in any circuit, not all parts run at the same time. Some paths execute only when a particular branch is taken; others sit idle.
- The Power Scheduler finds these mutually exclusive paths and groups them into **partitions**. A partition that is not needed at a given moment is switched off by a **guard signal** (gated clock, guarded evaluation, or power-down).
- Switching off unused hardware reduces the number of transistors switching per clock cycle, which directly lowers dynamic power consumption.
- Grouping several paths into one partition (instead of one partition per path) keeps the number of guard units small, because each guard unit itself consumes power.

III. ALGORITHM & SCHEDULING FLOW

Input: a Control-Data Flow Graph (CDFG), which combines a Control-Flow Graph (CFG) for decisions and a Data-Flow Graph (DFG) for computations. Output: the same graph split into guarded partitions.

TABLE I
STEPS OF THE POWER SCHEDULER

Step	Action	What it does
1	Parse CDFG	Read the CFG and DFG into memory
2	ASAP / ALAP	Schedule every operation as early and as late as possible; the gap is the <i>mobility</i> (flexibility) of each op
3a	Fork/join paths	Trace all paths between fork nodes and join nodes in the DFG – these paths are alternatives at runtime
3b	Control paths	Schedule control nodes as early as possible to expose which input branches can be deactivated
4	Compat. Graph	Build a graph where each node is a path; add an edge only if two paths have no shared fork/join/control node
5	Clique search	Find the largest groups (cliques) of compatible paths – each clique becomes one partition
6	Add guards	Attach an activation/deactivation control unit to each partition

Cost Function

The power cost of a single node (gate) is:

$$P_{\text{node}} = \frac{1}{2} C_L V_{dd}^2 \quad (1)$$

Including the clock frequency, the dynamic power of that node is:

$$P_{\text{dyn,node}} = \frac{1}{2} C_L V_{dd}^2 f_{clk} \quad (2)$$

The power-delay product for a single partition k , including the cost of its guard unit, is:

$$PD_k = (P_{\text{dyn},k} \cdot d_k) + (P_{gc,k} \cdot d_l) \quad (3)$$

The total cost that the Power Scheduler tries to minimise is the sum over all partitions l :

$$\text{TotalPD} = \sum_{x=1}^l \text{PD}_x \quad (4)$$

Here $P_{gc,k}$ is the power of the guard unit, d_k is the delay of partition k , and d_l is the delay of the surrounding circuit.

IV. ASSUMPTIONS & SCOPE LIMITATIONS

TABLE II
KEY ASSUMPTIONS AND LIMITATIONS

Item	Explanation
Pre-scheduled input	The Power Scheduler receives a CDFG that has already been scheduled; it does not schedule from scratch
Register before module allocation	Unlike most HLS tools, register allocation happens before functional unit allocation
Control-heavy designs	The approach is most effective for designs with many sequential branches and decision points
Self-controlled architectures	Targets designs with no global controller; each partition has its own local control logic
Guard overhead always present	Every partition adds a guard unit that is always on; this cost must be smaller than the saving
Leakage only partly reduced	Sharing hardware resources lowers leakage by reducing total transistor count; no voltage scaling is applied
Single benchmark only	Only one example (MPEG-2 chrominance conversion) is evaluated; how well results transfer to other designs is unknown

V. RESOURCE, RUNTIME & SCALABILITY

- **Clique search is NP-hard** in general. For very large designs the search could become slow; the paper does not discuss approximation strategies.
- The Compatibility Graph has at most $\mathcal{O}(p^2)$ edges, where p is the number of identified paths. This is manageable for small to medium designs.
- Scheduling control nodes early (Step 3b) adds one extra clock step; the effect on overall latency is not quantified.
- **Measured result:** 15 % energy reduction compared to an un-partitioned design on the MPEG-2 example. This is also better than a naive one-partition-per-path approach.
- Results were confirmed using Synopsys Power Compiler on synthesisable VHDL, which gives confidence in the numbers.

VI. OPEN QUESTIONS

- How is the NP-hard clique search handled for large CDFGs? Is an approximate algorithm used in practice?
- What determines the choice of guard type: gated clock, guarded evaluation, or power-down?

- Could per-partition voltage scaling (DVFS) be added on top of this partitioning to save even more energy?
- How large is the latency penalty from scheduling control nodes one step earlier, especially in tight timing budgets?
- The 15 % saving comes from one benchmark – would the result hold for control-light or data-dominated designs?

REFERENCES

- [1] A. Rettberg, C. Romaus, and H. Franke, “A new design partitioning approach for low power high-level synthesis,” in *Proc. IEEE*, 2006. [Online]. Available: <https://ieeexplore.ieee.org/document/1581203>